

Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Задание № 2

По дисциплине

“Алгоритмы компьютерной графики”

Выполнил:
Стрельбицкий Илья Павлович

Группа: Р3413

Преподаватель: Андреев Артем Станиславович

Санкт-Петербург, 2025г

Задание

1. Учимся выводить текстуру треугольника.

Показать в отчете passthrough координаты в нормализованной системе координат, помним, что деление на w осуществляет ускоритель сам. И показать, реализовав $0.5 * xy + \text{vec2}(0.5)$ как они пробегают все значения на экране

2. Выводим UV координаты
3. Выводим текстуру на экран, обязательно свою, а не всяких «чертей» из директории, сопровождающих NV примеры.

Ход работы

1. Задание №1

Исходный код с passthrough:

```
// Матрица для преобразования вершин
float4x4 WorldViewProj : WorldViewProjection;

// Структуры для передачи данных
struct VS_INPUT {
    float3 Pos : POSITION;
    float2 Tex : TEXCOORD0;
};

struct VS_OUTPUT {
    float4 Pos : POSITION;
    float2 TexCoord : TEXCOORD0;
    float4 ScreenPos : TEXCOORD1;
};

// Вершинный шейдер
VS_OUTPUT VS_Main(VS_INPUT input)
{
    VS_OUTPUT output;
    // Умножаем на матрицу, чтобы вершины попали на экран
    output.Pos = mul(float4(input.Pos, 1.0), WorldViewProj);
    output.TexCoord = input.Tex;
    output.ScreenPos = output.Pos; // Сохраняем спроецированную позицию
    return output;
}

// Пиксельный Шейдер
float4 PS_Main(VS_OUTPUT input) : COLOR
{
    // === ЗАДАНИЕ 1: Passthrough координаты ===
    // Деление на w для получения нормализованных координат (NDC)
    float2 ndc = input.ScreenPos.xy / input.ScreenPos.w;
    // Преобразование NDC [-1,1] в диапазон цвета [0,1]
    float2 remappedCoords = ndc * 0.5 + 0.5;
    // Возвращаем как цвет (Red = X, Green = Y)
    return float4(remappedCoords, 0.0, 1.0);
}

technique technique0
{
    pass p0
    {
        CullMode = None; // Отключаем отсечение граней, чтобы все было видно
```

```

VertexShader = compile vs_3_0 VS_Main();
PixelShader = compile ps_3_0 PS_Main();
}
}

```

Получаем следующий результат:

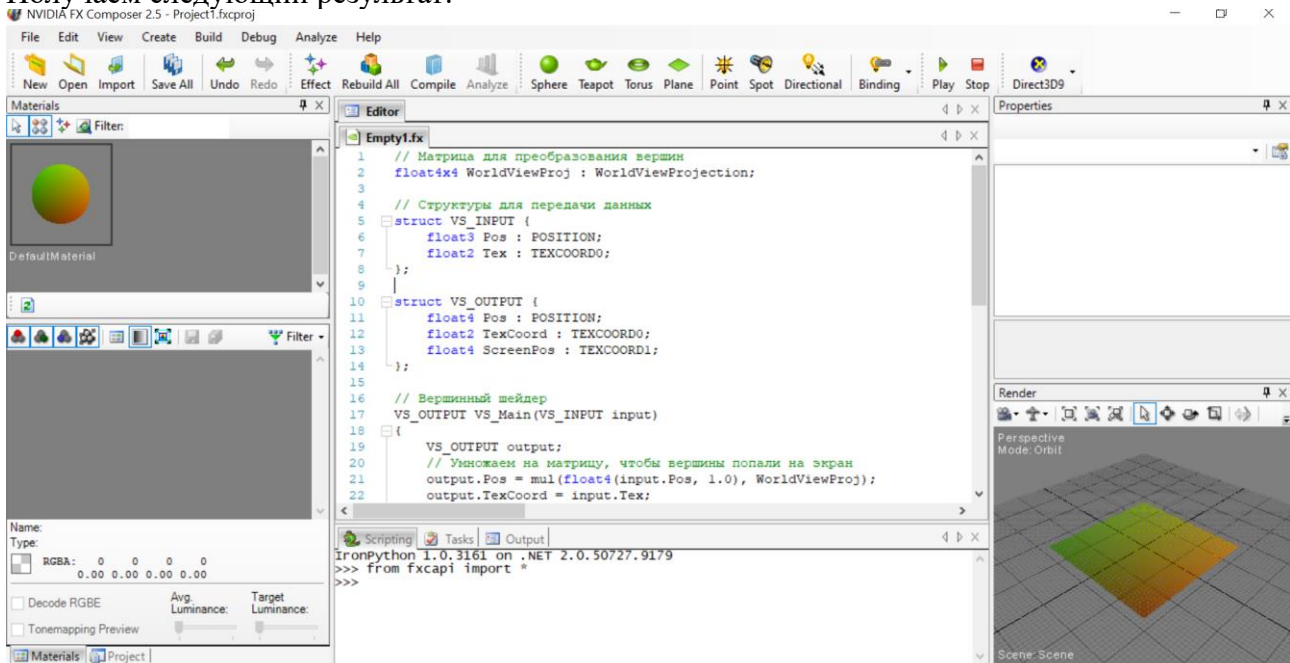


Рисунок 1. Эффект с passthrough координатами

2. Задание № 2

Модифицируем код из задания 1.

В пиксельном шейдере меняем возвращаемое значение с:

```
return float4(remappedCoords, 0.0, 1.0);
```

На:

```
return float4(input.TexCoord, 0.0, 1.0);
```

Получаем следующий результат:

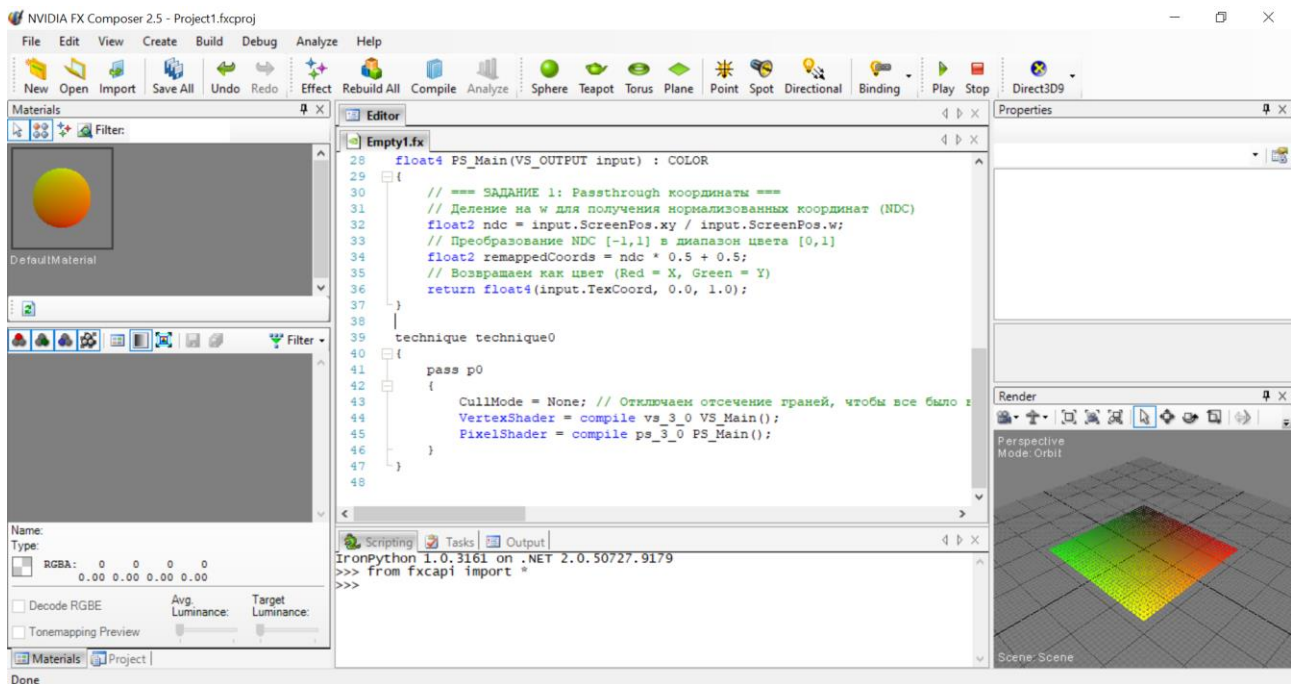


Рисунок 2. Эффект с UV координатами

3. Задание №3

Добавляем в начало файла:

```
texture MyTexture; // Объявляем текстуру
sampler2D MySampler = sampler_state // Объявляем сэмплер
{
    Texture = <MyTexture>;
    MinFilter = Linear;
    MagFilter = Linear;
};
```

И в пиксельном шейдере меняем возвращаемое значение на:

```
return tex2D(MySampler, input.TexCoord);
```

Затем в панели свойств материала в параметре MyTexture загружаем наше изображение.

Получившийся исходный код:

```
// Матрица для преобразования вершин
float4x4 WorldViewProj : WorldViewProjection;

texture MyTexture; // Объявляем текстуру
sampler2D MySampler = sampler_state // Объявляем сэмплер
{
    Texture = <MyTexture>;
    MinFilter = Linear;
    MagFilter = Linear;
};

// Структуры для передачи данных
struct VS_INPUT {
    float3 Pos : POSITION;
    float2 Tex : TEXCOORD0;
};

struct VS_OUTPUT {
    float4 Pos : POSITION;
    float2 TexCoord : TEXCOORD0;
```

```

float4 ScreenPos : TEXCOORD1;
};

// Вершинный шейдер
VS_OUTPUT VS_Main(VS_INPUT input)
{
    VS_OUTPUT output;
    // Умножаем на матрицу, чтобы вершины попали на экран
    output.Pos = mul(float4(input.Pos, 1.0), WorldViewProj);
    output.TexCoord = input.Tex;
    output.ScreenPos = output.Pos; // Сохраняем спроецированную позицию
    return output;
}

// Пиксельный Шейдер
float4 PS_Main(VS_OUTPUT input) : COLOR
{
    // === ЗАДАНИЕ 1: Passthrough координаты ===
    // Деление на w для получения нормализованных координат (NDC)
    float2 ndc = input.ScreenPos.xy / input.ScreenPos.w;
    // Преобразование NDC [-1,1] в диапазон цвета [0,1]
    float2 remappedCoords = ndc * 0.5 + 0.5;
    // Возвращаем как цвет (Red = X, Green = Y)
    return tex2D(MySampler, input.TexCoord);
}

technique technique0
{
    pass p0
    {
        CullMode = None; // Отключаем отсечение граней, чтобы все было видно
        VertexShader = compile vs_3_0 VS_Main();
        PixelShader = compile ps_3_0 PS_Main();
    }
}

```

Получившийся результат:

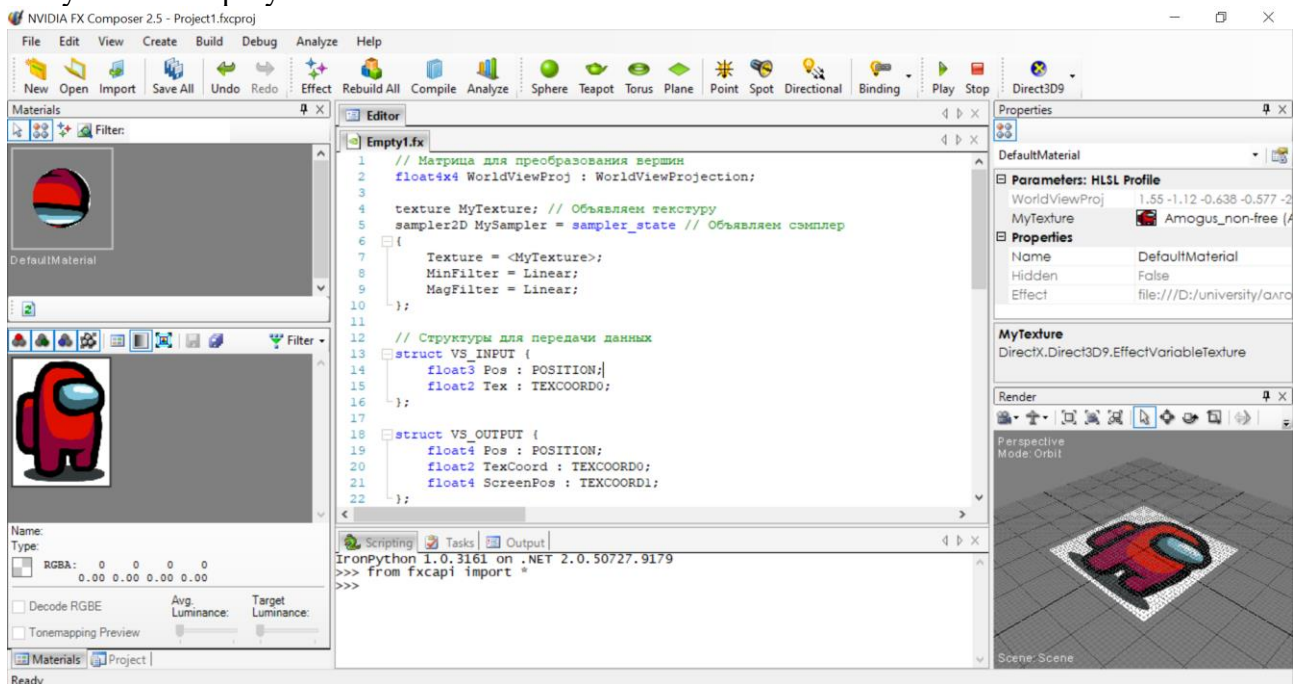


Рисунок 3. Эффект с текстурой